AI Structural-Break Detector for Statistical Arbitrage

Data Architecture and Source Strategy

Postgraduate group project - Track A (Tech Team)

Complete data specification: source contracts, ingestion, storage, lineage, point-in-time controls, and app-facing datasets

Data design principle

Raw source snapshots are immutable. Every processed row must be traceable back to a source snapshot. Historical backtests and historical agent investigations may use only information that was actually available by the event timestamp.

System boundary

The historical research pipeline is built offline and frozen into reproducible snapshots. The Streamlit application reads local curated data and analytics; it does not depend on live web calls to render historical results. Live/current refreshes are a separate optional job.

Source contracts verified: 28 September 2026

Contents

1. Purpose, scope, and non-negotiable data rules
2. Architecture at a glance
3. Canonical source registry
4. Reference data: securities and pair universe
5. Equity price and corporate-action data
6. Market and regime data
7. SEC filing metadata and document evidence
8. Optional historical/current news enrichment
9. Exact storage layout and naming conventions
10. Curated datasets and DuckDB warehouse
11. Core schemas: fields that must be stored
12. Point-in-time joins and leakage prevention
13. Ingestion and refresh jobs
14. Data quality, validation, and lineage
15. How the application reads the data
16. Deployment and demo-data packaging
17. Build order and acceptance criteria
Appendix A. Retrieval endpoints and source rules
Appendix B. IDs, timestamps, and file naming

Relationship to the main project overview

This document expands only the Data Architecture and Source Strategy layer. It keeps the existing project scope: US large-cap equity pairs, daily end-of-day data, a deterministic statistical relationship engine, an event-based ML dataset, and public evidence for structural-break investigation.

1. Purpose, scope, and non-negotiable data rules

1.1 What the data layer must support

The data architecture must provide a reproducible, point-in-time dataset for five downstream jobs: pair relationship estimation, divergence-event construction, supervised ML feature generation, structural-break evidence retrieval, and historical strategy evaluation. It also has to support a polished app without requiring network access during a demo.

1.2 Final working data scope

Item	Working specification
Asset universe	US-listed domestic large-cap equities only; 20-40 manually curated same-sector or same-sub-industry pairs.
Frequency	Daily end-of-day data. No intraday order-book or tick data.
Raw price history	Recommended download starts 2014-01-01 and freezes at 2025-12-31. This supplies roughly two years of warm-up before a 2016 research window when using a 504-trading-day formation window.
Market context	SPY, one sector ETF per security, and official Cboe VIX history.
Canonical text evidence	SEC EDGAR filing metadata and selected filing documents.
Optional text enrichment	GDELT article metadata/current news only; not required for headline historical evaluation.
Storage engine	Immutable raw files + curated Parquet + DuckDB analytics warehouse.
App access	Read-only queries against DuckDB/Parquet. External APIs are called by ingestion jobs, not by historical UI pages.

1.3 Non-negotiable rules

- Every network retrieval receives a UTC retrieval timestamp and a source identifier.
- Raw downloads are never overwritten. Corrections create a new snapshot.
- No forward filling of market prices across non-trading days or missing observations.
- Features are computed from trailing/current observations only; future data is reserved for label construction.
- For document evidence, the SEC acceptance timestamp is the governing availability time.
- Historical results are tied to a frozen data_snapshot_id so the same model can be reproduced later.
- Optional news cannot affect the headline historical result unless timestamp coverage and document access are demonstrably point-in-time.

2. Architecture at a glance

2.1 Five-zone architecture

Layer	Purpose	Exact storage
1. Source systems	External public data providers and manually curated reference files.	Yahoo Finance via yfinance; SEC EDGAR; Cboe; optional GDELT; versioned project config.
2. Raw zone	Immutable source snapshots. No modelling transformations.	data/raw/... in provider-native or source-like files: JSON, HTML/TXT, CSV, Parquet snapshots.
3. Curated zone	Clean, typed, deduplicated, aligned datasets used by research code.	data/curated/... as Parquet.
4. Analytics warehouse	Pair estimates, divergence events, labels, predictions, agent outputs, decisions, quality logs.	data/warehouse/structural_break.duckdb.
5. Application bundle	Frozen subset required for the deployed demo.	data/demo/... with a read-only DuckDB file plus compact Parquet/document assets.

2.2 Data flow

SOURCE -> immutable RAW snapshot -> validation/normalisation -> CURATED Parquet -> ANALYTICS in DuckDB -> Streamlit UI

The app never treats an external provider as the database. External providers are ingestion sources. Once downloaded, the project works from its own versioned snapshots so backtests do not silently change when a vendor revises history.

2.3 Why DuckDB + Parquet

- Parquet is compact, typed, columnar, and easy to inspect or move between machines.
- DuckDB queries Parquet directly and is sufficient for a course-scale analytical warehouse without running a database server.
- The combination supports SQL joins, time-window queries, app filtering, and reproducible local execution.
- If the project were productionised, the same logical schemas could migrate to a managed warehouse without changing the data contracts.

3. Canonical source registry

Dataset ID	Source	Retrieval interface	Core?	Used for
ref.security_master	Manual project file + SEC ticker/CIK mapping	Versioned CSV/Parquet + SEC company_tickers.json	Yes	Ticker identity, CIK, exchange, sector, sector ETF, lifecycle.
market.equity_daily	Yahoo Finance via yfinance	yfinance.download, daily interval	Yes	Prices, adjusted prices, volume, returns, spread, liquidity.
market.actions	Yahoo Finance via yfinance	Same daily download with actions enabled	Yes	Dividends and stock splits for audit/corporate-action checks.
market.benchmark_daily	Yahoo Finance via yfinance	SPY + sector ETFs using same daily contract	Yes	Broad-market and sector returns/regime features.
market.vix_daily	Cboe	Official historical VIX CSV	Yes	Market stress / volatility features.
evidence.sec_index	SEC EDGAR	data.sec.gov submissions JSON	Yes	Filing availability, accession IDs, form, acceptance timestamp.
evidence.sec_documents	SEC EDGAR Archives	Archives filing directory / documents	Yes	Canonical public text evidence for agent investigation.
news.gdelt	GDELT Article List / GDELT data	Public JSON/BigQuery/feeds	Optional	Additional current or timestamped article discovery; enrichment only.

Source hierarchy

For historical agent evaluation, SEC EDGAR is authoritative. Yahoo/yfinance is the prototype price source, not a production market-data feed. GDELT is enrichment only. This hierarchy prevents the project from making strong claims based on news coverage that may be incomplete or unreproducible.

3.1 Authentication and secrets

Source	API key required?	Credential handling
yfinance / Yahoo Finance	No	No secret. Record yfinance package version in the manifest.
SEC EDGAR	No	No key. A declared User-Agent with project/team contact is required for scripted access; obey SEC fair-access limits.
Cboe VIX history	No	No secret.
GDELT	No for public datasets	No secret for public files/BigQuery public dataset access; Google Cloud billing/project may be needed for BigQuery beyond free allowance.

4. Reference data: securities and pair universe

4.1 security_master - manually governed, not inferred at runtime

The pair universe must not depend on a live screener. The team should create and review a versioned security master once, because ticker mappings, sector classifications, and business similarity are part of the research design.

Field	Type / example	Why it is stored
security_id	string; US:AAPL	Stable internal key even if a display ticker later changes.
ticker	string; AAPL	Market-data symbol.
issuer_name	string	Human-readable identity.
cik	10-digit string	SEC EDGAR lookup key.
exchange	string; NASDAQ	Reference/audit field.
sector	controlled string	Economic pre-filter and reporting.
sub_industry	controlled string	Pair plausibility and robustness reporting.
sector_etf	string; XLK	Sector benchmark used for context features.
active_from / active_to	date / nullable	Security lifecycle and point-in-time validity.
domestic_sec_issuer	boolean	MVP requires true; foreign private issuers can be excluded.
source_snapshot_id	string	Lineage to SEC mapping/reference version.
manual_review_status	APPROVED / REJECTED	Prevents accidental automatic pair creation.

4.2 pair_universe

Field	Example / rule
pair_id	V__MA using a canonical ticker ordering rule.
leg_a_security_id / leg_b_security_id	Foreign keys to security_master.
sector / sub_industry	Must be economically consistent with both legs.
economic_rationale	One or two sentences explaining why the pair is comparable.
enabled	Boolean switch controlled by config.
pair_version	Version string for the curated universe.
created_at / reviewed_at	Audit timestamps.
notes	Ticker changes, mergers, spin-offs, known complications.

4.3 Exact retrieval of ticker -> CIK

Download the SEC mapping file from the official Company Tickers endpoint, store the raw JSON snapshot, normalize ticker/CIK/company name into the security master, and manually reconcile any unmapped or changed tickers. The SEC itself notes that the mapping is periodically updated and does not guarantee complete accuracy, so the project should not treat it as a historical sector database.

https://www.sec.gov/files/company_tickers.json

5. Equity price and corporate-action data

5.1 Prototype retrieval contract

Parameter	Exact setting
Library	yfinance, pinned in requirements and recorded in every snapshot manifest.
Function	Multi-ticker historical download through yfinance.download.
Interval	1d.
Start	2014-01-01 recommended for the frozen research dataset.
End	2026-01-01 for a full through-2025 snapshot; yfinance treats end as exclusive.
auto_adjust	False, so both raw OHLC and Adj Close remain observable.
actions	True, so dividends and stock splits are captured.
repair	False for raw ingestion; any repair belongs in curated data with an explicit flag.
pre/post	False; daily regular-session data only.
Universe	All unique pair legs + SPY + required sector ETFs.
Refresh	Historical bootstrap once; optional daily refresh after market close for the current scanner.

5.2 Fields to preserve in raw price snapshots

Field	Storage rule
trading_date	Exchange trading date; DATE, not a UTC timestamp.
open/high/low/close	Vendor raw regular-session OHLC.
adj_close	Corporate-action-adjusted closing series used for pair modelling.
volume	Raw daily share volume.
dividends	Cash dividend amount if non-zero.
stock_splits	Split ratio/event value if non-zero.
ticker	Requested vendor symbol.
retrieved_at_utc	When the snapshot was obtained.
source_provider	YAHOO_YFINANCE.
source_snapshot_id	Immutable snapshot key.
library_version	Pinned yfinance version used for retrieval.

5.3 Exact file locations

data/raw/yahoo/prices/{ticker}/{snapshot_id}.parquet

data/raw/yahoo/actions/{ticker}/{snapshot_id}.parquet

data/curated/market/prices_daily.parquet

data/curated/market/corporate_actions.parquet

Important prototype limitation

Yahoo/yfinance is suitable for a reproducible academic prototype but is not a licensed institutional market-data feed. Any production version would replace the ingestion adapter with a licensed vendor while retaining the same curated schema.

6. Market and regime data

6.1 Broad market and sector benchmarks

SPY and sector ETFs use exactly the same yfinance daily retrieval contract as equities. They therefore share the same calendar, source metadata, and raw/curated schema.

Sector label	Benchmark symbol
Communication Services	XLC
Consumer Discretionary	XLY
Consumer Staples	XLP
Energy	XLE
Financials	XLF
Health Care	XLV
Industrials	XLI
Information Technology	XLK
Materials	XLB
Real Estate	XLRE
Utilities	XLU

6.2 VIX - authoritative volatility series

Use the official Cboe historical VIX file. Preserve the original CSV unchanged in the raw zone, then normalize date and OHLC fields into the curated market-context table.

https://cdn.cboe.com/api/global/us_indices/daily_prices/VIX_History.csv

Raw VIX field	Curated field
DATE	trading_date
OPEN	vix_open
HIGH	vix_high
LOW	vix_low
CLOSE	vix_close

6.3 Curated market context

Field	Meaning
trading_date	NYSE session date.
spy_adj_close / spy_return	Broad market level and return.
sector_etf_adj_close / sector_return	Pair-specific sector benchmark.
vix_close	Market-implied volatility level.

Field	Meaning
market_realised_vol_20d	Trailing market realised volatility.
sector_realised_vol_20d	Trailing sector realised volatility.
data_available_at	Timestamp at which the row is admissible to the feature pipeline.

data/raw/cboe/vix/{snapshot_id}_VIX_History.csv

data/curated/market/market_context_daily.parquet

7. SEC filing metadata and document evidence

7.1 Why SEC is the canonical historical evidence source

EDGAR provides reproducible filing identifiers, official acceptance timestamps, company CIKs, and archival document paths. That makes it materially safer for point-in-time agent evaluation than a general web-news search.

7.2 Step 1 - retrieve filing metadata

For every CIK in security_master, request the SEC submissions JSON. The current submissions endpoint contains recent filing arrays and may reference additional JSON files for older history. Store the raw response exactly as received before normalization.

https://data.sec.gov/submissions/CIK##########.json

Metadata field	Store?	Use
accessionNumber	Yes	Primary filing identifier / filing_id.
form	Yes	8-K, 10-Q, 10-K filter.
filingDate	Yes	Calendar filing date.
reportDate	Yes	Period covered; not the availability timestamp.
acceptanceDateTime	Yes - critical	Point-in-time evidence cutoff.
primaryDocument	Yes	Construct primary document URL.
primaryDocDescription	Yes	Document metadata.
items	Yes	Useful for 8-K item context where available.
size	Yes	Download planning / audit.
isInlineXBRL / isXBRL	Yes	Parsing metadata.
fileNumber / filmNumber	Optional	Audit/reference only.

7.3 Step 2 - normalize the filing index before downloading bodies

Normalize all relevant filing metadata into sec_filing_index first. Do not download every document body. This index allows the event pipeline to select only filings that are actually needed by a historical divergence event.

data/raw/sec/submissions/{cik}/{snapshot_id}.json

data/curated/evidence/sec_filing_index.parquet

7. SEC filing metadata and document evidence (continued)

7.4 Step 3 - evidence selection policy

Document class	Selection rule at divergence time t
Recent material-event filings	All 8-K filings accepted in the 30 calendar days ending at t.
Periodic context	Most recent 10-Q or 10-K accepted on or before t, normally within the prior 180 days; extend to 400 days if necessary.
Amendments	Include 8-K/A, 10-Q/A, 10-K/A only if accepted before t; tag as amendment.
Foreign issuer forms	Not required in MVP because domestic SEC issuers are the intended universe. If later added, 6-K and 20-F require the same timestamp rules.

7.5 Step 4 - construct archive URLs

For an accession number, remove dashes to build the filing folder and remove leading zeros from the CIK for the archive path. Download the filing directory index, the primary filing, and the complete submission text. The complete submission is particularly useful because it contains embedded document types and exhibits such as EX-99.1 earnings releases.

Folder: https://www.sec.gov/Archives/edgar/data/{CIK_unpadded}/{accession_no_dashes}/

Directory listing: .../{accession_no_dashes}/index.json

Primary document: .../{accession_no_dashes}/{primaryDocument}

Complete submission: .../{accession_no_dashes}/{accession_with_dashes}.txt

7.6 SEC network rules

- Declare a project/team User-Agent and contact email in every scripted request.
- Keep aggregate request rate below the SEC fair-access ceiling of 10 requests per second; the project should target materially lower throughput (for example 2-5 requests/second) with retries/backoff.
- Cache by accession number; never re-download the same filing during ordinary pipeline reruns.
- Persist HTTP status, source URL, bytes, retrieval timestamp, and SHA-256 hash in the ingestion manifest.

7. SEC filing metadata and document evidence (continued)

7.7 Exact raw and curated locations

data/raw/sec/company_tickers/{snapshot_id}.json

data/raw/sec/submissions/{cik}/{snapshot_id}.json

data/raw/sec/submissions/{cik}/history/{sec_history_filename}.json

data/raw/sec/filings/{cik}/{accession}/index.json

data/raw/sec/filings/{cik}/{accession}/complete_submission.txt

data/raw/sec/filings/{cik}/{accession}/primary_document.html

data/raw/sec/filings/{cik}/{accession}/exhibits/{sequence}_{type}_{filename}

data/curated/evidence/sec_filing_index.parquet

data/curated/evidence/document_chunks.parquet

7.8 Document parsing and chunk storage

Field in document_chunks	Purpose
chunk_id	Stable hash/ID for a filing section or exhibit chunk.
filing_id / accession	Traceability to SEC filing.
security_id	Company identity.
form	8-K / 10-Q / 10-K.
document_type	Primary form, EX-99.1, EX-99.2, etc.
acceptance_datetime_utc	Availability timestamp used by point-in-time filtering.
section_label	Detected heading / item where available.
chunk_order	Original ordering inside document.
chunk_text	Cleaned evidence text.
char_count / token_estimate	Context-window planning.
source_url	Original SEC document URL.
raw_file_path	Local immutable source file.
sha256	Integrity / lineage.

Core retrieval for the agent should first filter by security_id, acceptance timestamp, and document type; only then rank the resulting chunks. This avoids searching the entire filing corpus for every event.

8. Optional historical/current news enrichment

8.1 Scope

News is useful for catalysts that may not yet appear in filings, but historical article availability, paywalls, corrections, duplicated syndication, and publication timestamps are harder to defend than EDGAR. Therefore news is not required for the primary historical result.

8.2 Preferred optional source

Use GDELT only as an article-discovery/enrichment layer. The GDELT Article List provides article URLs/titles and public archive/BigQuery access. Store the GDELT metadata and the original publication URL; if article text is fetched, store it only when the page is publicly accessible and preserve the retrieval timestamp.

GDELT Article List archive pattern: https://data.gdeltproject.org/gdeltv3/gal/YYYYMMDDHHMMSS.gal.json.gz

BigQuery table: gdelt-bq.gdeltv2.gal

8.3 Fields to store when news is enabled

Field	Rule
news_id	Stable hash from normalized URL + publication timestamp.
security_id / matched_entity	Company match used by retrieval.
title	Article title from discovery source.
publisher_domain	Source domain.
published_at	Original/source publication time if available; never replace with retrieval time.
gdelt_seen_at	When GDELT observed the article.
url	Original article URL.
article_text	Optional; only if legally/publicly accessible and successfully fetched.
retrieved_at_utc	When the project fetched metadata/text.
point_in_time_eligible	True only when published_at <= event cutoff and timestamp quality is acceptable.
source_quality_flag	HIGH / MEDIUM / LOW / UNKNOWN.

Evaluation rule

If timestamp quality or article availability cannot be defended, the news record may be shown as demo context but cannot enter historical ML features or the headline agent-veto evaluation.

9. Exact storage layout and naming conventions

9.1 Repository-relative data tree

```
data/
  reference/
    security_master.parquet
    pair_universe.parquet
    sector_etf_map.parquet
  raw/
    yahoo/prices/{ticker}/{snapshot_id}.parquet
    yahoo/actions/{ticker}/{snapshot_id}.parquet
    cboe/vix/{snapshot_id}_VIX_History.csv
    sec/company_tickers/{snapshot_id}.json
    sec/submissions/{cik}/{snapshot_id}.json
    sec/filings/{cik}/{accession}/...
    gdelt/{YYYY}/{MM}/{DD}/...          # optional
  curated/
    market/prices_daily.parquet
    market/corporate_actions.parquet
    market/market_context_daily.parquet
    evidence/sec_filing_index.parquet
    evidence/document_chunks.parquet
    evidence/news_index.parquet         # optional
  warehouse/structural_break.duckdb
  snapshots/{data_snapshot_id}/manifest.json
  demo/...
```

9.2 Git policy

- Commit reference/config files and schema definitions to Git.
- Exclude full raw data and large curated datasets from ordinary Git history.
- Commit only a compact demo bundle if repository limits permit; otherwise publish the bundle as a release asset/object-store artefact.
- Never commit API secrets; the chosen core sources do not require secrets.

10. Curated datasets and DuckDB warehouse

10.1 Curated Parquet files - canonical cleaned data

File	Grain	Primary key
prices_daily.parquet	One security x trading day	security_id + trading_date
corporate_actions.parquet	One action event	security_id + action_date + action_type
market_context_daily.parquet	One trading day x sector benchmark	trading_date + sector_etf
sec_filing_index.parquet	One SEC filing	accession_number
document_chunks.parquet	One clean text chunk	chunk_id
news_index.parquet	One news article (optional)	news_id

10.2 DuckDB schemas

DuckDB schema	Objects
ref	Views/tables for securities, pair universe, sector ETF mapping, configuration versions.
curated	Views over Parquet for prices, market context, filing index, document chunks, optional news.
analytics	Physical tables for pair estimates, divergence events, features, labels, model predictions, agent findings, decisions, and backtest trades.
ops	Ingestion runs, snapshot manifests, data-quality results, and source-health checks.

10.3 Why not store raw documents in DuckDB

SEC HTML/TXT files are immutable source artefacts and are easier to audit as files. DuckDB stores their metadata, local path, hash, and parsed chunks. This keeps the analytical database compact while preserving original evidence.

11. Core schemas: fields that must be stored

11.1 prices_daily

Field	Type / meaning
security_id	string
trading_date	date
open/high/low/close	double
adj_close	double
volume	integer
daily_log_return	double; derived trailing/current only
dollar_volume	double; liquidity proxy
source_snapshot_id	string
retrieved_at_utc	timestamp
quality_flag	OK / WARN / FAIL

11.2 sec_filing_index

Field	Type / meaning
filing_id	accession number
security_id / cik	identity
form	8-K / 10-Q / 10-K / amendment
filing_date	date
report_date	date
acceptance_datetime_utc	timestamp; point-in-time gate
items	text/list
primary_document	filename
primary_url	URL
complete_submission_url	URL
raw_folder_path	local path
download_status	NOT_REQUIRED / QUEUED / DOWNLOADED / FAILED
source_snapshot_id	string

11.3 divergence_events - data-layer view of an event

Field	Meaning
event_id	Stable event key.
pair_id	Pair identity.
event_date / event_timestamp	End-of-day research timestamp.
direction	Positive/negative divergence state.
entry_zscore	Observed divergence state at event time.
pair_estimate_id	Exact rolling relationship estimate used.

Field	Meaning
data_snapshot_id	Frozen source snapshot.
evidence_cutoff	Latest admissible document timestamp.
resolution_status	OPEN / SUCCESS / FAIL / TIMEOUT; label generated later.

11. Core schemas: fields that must be stored (continued)

11.4 event_features

Feature family	Examples stored per event
Spread state	z-score, spread velocity, distance from equilibrium, recent crossings.
Relationship quality	cointegration test statistic/p-value, half-life estimate, hedge-ratio stability, rolling correlation.
Market/regime	SPY return, sector return, VIX, trailing realised volatility, optional regime state.
Security state	relative momentum, abnormal volume, relative volatility, price gap indicators.
Liquidity	average dollar volume, volume imbalance, simple cost/turnover proxies.
Event/NLP	event_type, affected_leg, materiality, evidence_count, document_recency, extraction confidence; only if point-in-time eligible.
Lineage	feature_schema_version, generated_at_utc, data_snapshot_id.

11.5 agent_findings and decisions

Table	Required fields
agent_findings	event_id, agent_name, model_name/version, prompt_version, finding_type, structured JSON payload, source_ids, source_cutoff, confidence, run_timestamp, validation_status.
decisions	event_id, ml_probability, break_risk, liquidity_gate, evidence_gate, data_quality_gate, final_status, reason_codes, controller_version, decided_at_utc.

11.6 ingestion_runs

Every ingestion job writes one operational record: run_id, dataset_id, started_at, finished_at, source endpoint, request parameters, source version/library version, files written, row count, min/max date, HTTP/result status, checksum summary, warnings, and data_snapshot_id.

12. Point-in-time joins and leakage prevention

12.1 Daily market timestamp policy

A divergence event based on trading day t closing data is considered known only after the regular US market close. Any simulated position based on that close must therefore enter no earlier than the next tradable session. This avoids using a close price and pretending the order was executed before that close was known.

12.2 Document availability policy

A filing is admissible only when its SEC acceptance timestamp is less than or equal to the event evidence cutoff. reportDate is not an availability timestamp. Filings accepted after the cutoff move to the next decision period.

12.3 Point-in-time join rules

Join	Allowed rule
Event -> market context	Use same-day close-derived values only after close; all rolling features end at t.
Event -> pair estimate	Use the most recent estimate whose effective date is <= t and whose formation window ends before or at t.
Event -> SEC filing	acceptance_datetime <= evidence_cutoff.
Event -> news	published_at <= evidence_cutoff AND timestamp quality accepted.
Event -> label	Future spread path is accessed only by the label builder, never by feature generation.
Training -> validation/test	Chronological partition; no random leakage across future periods.

12.4 Automated leakage assertions

- max(feature_source_timestamp) <= event_timestamp for every event row.
- No event feature row references a filing accepted after the event cutoff.
- No rolling feature window includes dates later than the event date.
- Label columns are not present in the feature-generation input frame.
- Final test events are excluded from threshold/pair/hyperparameter selection.

13. Ingestion and refresh jobs

Job	When	What it does	Writes
bootstrap_reference	Once, then manual updates	Build security master, pair universe, CIK map, sector ETF map.	reference/*
ingest_market_history	Initial bootstrap; manual rerun for new snapshot	Download all pair legs + SPY + sector ETFs with frozen date bounds.	raw/yahoo/*
ingest_vix_history	With market bootstrap	Download official Cboe VIX CSV.	raw/cboe/vix/*
normalize_market	After market ingestion	Validate, align trading calendar, build curated price/context tables.	curated/market/*
ingest_sec_index	Initial bootstrap + optional daily refresh	Download submissions metadata for each CIK, including older referenced JSON.	raw/sec/submissions/* + curated filing index
build_divergence_events	After clean market data	Run rolling pair logic and create event timestamps.	analytics.divergence_events
select_sec_evidence	After events exist	Select filing IDs within defined evidence windows.	ops download queue
download_sec_documents	On selected filings only	Fetch filing index, primary document, complete submission, relevant exhibits.	raw/sec/filings/*
parse_documents	After downloads	Clean/chunk text; persist provenance.	curated/evidence/ document_chunks.parquet
freeze_snapshot	Before model training / final demo	Hash files, record versions, create immutable manifest.	snapshots/{id}/manifest.json

13.1 Suggested current-scanner refresh

For a non-live course demo, one daily refresh after the US close is sufficient: update market data, Cboe VIX, SEC filing metadata, and then recompute current pair states. Historical pages should continue to use the frozen research snapshot rather than mixing in newly revised data.

14. Data quality, validation, and lineage

14.1 Blocking market-data checks

Check	Fail condition / action
Primary key uniqueness	Duplicate security_id + trading_date -> FAIL.
Price validity	Non-positive OHLC/Adj Close -> FAIL unless documented vendor issue.
OHLC consistency	low > high or open/close outside impossible range -> FAIL/WARN.
Volume	Negative volume -> FAIL; zero volume -> WARN for liquid large caps.
Calendar completeness	Unexpected missing sessions above configured tolerance -> WARN/FAIL.
Split/dividend audit	Large raw-price jump without matching adjustment/action -> review.
Benchmark alignment	SPY/sector/VIX missing on required event date -> event evidence/feature may become UNKNOWN/ABSTAIN.

14.2 Blocking SEC checks

Check	Rule
CIK mapping	Every active security must have exactly one approved CIK.
Accession uniqueness	Accession number unique across filing index.
Acceptance timestamp	Required for point-in-time eligibility; missing -> not eligible.
Document hash	Stored file SHA-256 must match manifest on later reads.
HTTP status	Non-200 downloads stored as FAILED; no silent empty documents.
Source provenance	Every parsed chunk must reference raw_file_path + filing_id + source_url.

14.3 Snapshot manifest

The manifest is the reproducibility anchor. It lists source datasets, date coverage, source/library versions, every curated file hash, reference config version, data-quality summary, and creation timestamp. Model training stores the exact data_snapshot_id beside the model artefact.

15. How the application reads the data

15.1 Historical UI rule

Historical app pages do not call yfinance, SEC, Cboe, or GDELT. They open the frozen DuckDB database and curated Parquet files. This makes the demo fast, prevents provider outages from breaking the presentation, and ensures a historical event always shows the same inputs.

UI page	Data it reads
Relative-Value Scanner	Latest analytics.divergence_events + pair estimates + ML predictions + final decisions + ref pair metadata.
Pair Detail - charts	curated.prices_daily + analytics pair estimates/spread series.
Pair Detail - ML	analytics.event_features + predictions + SHAP outputs/model metadata.
Pair Detail - evidence	curated.sec_filing_index + document chunks referenced by agent_findings.
Pair Detail - agents	analytics.agent_findings + analytics.decisions.
Strategy Lab	analytics.backtest_trades + precomputed metric tables by baseline/model/regime/sector.
Model & Data Monitor	ops.ingestion_runs + data_quality_results + snapshot metadata + model version.

15.2 Current-event refresh

If the app supports a current scanner, the UI triggers or reads the output of a separate refresh job. The refresh writes a new source snapshot and analytics state; it never mutates the frozen training/test snapshot.

15.3 Caching

- Cache DuckDB connections and static reference tables in Streamlit.
- Cache pair-detail query results by pair_id + event_id + data_snapshot_id.
- Cache LLM/agent results by event_id + prompt_version + model_version + evidence_hash so repeated page loads do not re-run expensive calls.

16. Deployment and demo-data packaging

16.1 Local development

Artefact	Location / policy
Full raw history	Local data/raw; excluded from Git.
Full curated Parquet	Local data/curated; excluded from ordinary Git.
Research warehouse	Local data/warehouse/structural_break.duckdb.
Model artefacts	artifacts/models/{model_version}/.
Reference/config	Versioned in Git.
Snapshot manifest	Versioned or archived with experiment artefacts.

16.2 Deployed Streamlit demo

Build a compact read-only demo bundle containing only the final reference tables, price history required by displayed pairs/events, evidence chunks used by displayed events, analytics tables, and model metadata. The deployed app should start from this bundle without needing to download the research corpus.

data/demo/structural_break_demo.duckdb

data/demo/prices_demo.parquet

data/demo/document_chunks_demo.parquet

data/demo/demo_manifest.json

16.3 Production replacement path

A real trading deployment would replace Yahoo/yfinance with a licensed point-in-time market-data provider, use durable object storage and a managed database/warehouse, add market calendars and corporate-action mastering from the licensed feed, and implement monitored streaming/event ingestion. The prototype schemas are deliberately provider-neutral so this replacement does not require redesigning downstream models.

17. Build order and acceptance criteria

Phase	Deliverable	Acceptance criterion
1. Reference master	security_master + pair_universe + CIK mapping	Every active ticker has approved pair metadata, CIK, sector, sector ETF.
2. Market bootstrap	Raw Yahoo + Cboe snapshots	Frozen date coverage complete; manifests and hashes written.
3. Curated market layer	prices_daily + market_context_daily	No duplicate keys; calendar/price QA passes.
4. SEC index	Normalized filing metadata	8-K/10-Q/10-K history queryable by security and acceptance time.
5. Event build	Historical divergence events	Each event carries event_timestamp, pair_estimate_id, data_snapshot_id.
6. Evidence download	Only relevant SEC documents	Every selected filing cached once and hash-verified.
7. Document chunks	Parsed evidence dataset	Every chunk traces to accession, URL, timestamp, and raw path.
8. Snapshot freeze	Immutable research snapshot	Manifest can reproduce all model inputs.
9. App bundle	Read-only demo dataset	App runs with network disabled and historical pages remain functional.

Minimum viable data milestone

Before any XGBoost or LLM work starts, the team should be able to select a pair and a historical date and retrieve: the exact adjusted prices known at that date, the pair/market context, every eligible SEC filing accepted before the cutoff, and the raw source path/hash for each item.

Appendix A. Retrieval endpoints and source rules

Provider	Endpoint / interface	Verified use
yfinance	Python package: yfinance.download	Daily OHLCV/actions; start inclusive, end exclusive; auto_adjust=False; actions=True.
SEC ticker map	https://www.sec.gov/files/company_tickers.json	Ticker -> CIK/company mapping snapshot.
SEC submissions	https://data.sec.gov/submissions/CIK##########.json	Filing metadata, acceptance timestamps, older-history file pointers.
SEC archive	https://www.sec.gov/Archives/edgar/data/{CIK}/{accession_no_dashes}/	Primary filing, complete submission, exhibits, index.json.
Cboe VIX	https://cdn.cboe.com/api/global/us_indices/daily_prices/VIX_History.csv	Official historical daily VIX OHLC file.
GDELT Article List	https://data.gdeltproject.org/gdeltv3/gal/... and gdelt-bq.gdeltv2.gal	Optional article discovery / metadata enrichment.

A.1 Official access requirements that affect implementation

- SEC data APIs do not require authentication/API keys, but scripted requests must declare a User-Agent and comply with the current 10-requests/second fair-access ceiling.
- SEC submissions data is updated throughout the day and older filing history may be referenced through additional JSON files.
- Cboe publishes VIX historical data directly and updates the history file daily.
- yfinance is a third-party Python interface to Yahoo Finance data; its exact library version should be pinned and recorded because it is not the institutional source of truth.
- GDELT data is optional and should never silently substitute a retrieval timestamp for an unknown original publication time.

A.2 Source references

yfinance API reference: https://ranaroussi.github.io/yfinance/reference/api/yfinance.download.html

SEC EDGAR APIs: https://www.sec.gov/search-filings/edgar-application-programming-interfaces

SEC Accessing EDGAR Data / fair access: https://www.sec.gov/search-filings/edgar-search-assistance/accessing-edgar-data

SEC Company Tickers: https://www.sec.gov/file/company-tickers

Cboe VIX historical data: https://www.cboe.com/tradable_products/vix/vix_historical_data

GDELT Article List: https://blog.gdeltproject.org/announcing-the-gdelt-article-list-rss-feed/

Appendix B. IDs, timestamps, and file naming

Identifier	Format	Example / rule
data_snapshot_id	snap_YYYYMMDD_HHMMSSZ	One immutable research-data freeze.
source_snapshot_id	{provider}_{YYYYMMDD_HHMMSSZ}	One provider ingestion batch.
security_id	US:{ticker}	US:AAPL.
pair_id	{tickerA}__{tickerB}	Canonical ordering; stable across runs.
filing_id	SEC accession number	0000320193-25-000079.
chunk_id	hash(filing_id + document + order + text hash)	Stable evidence reference.
event_id	{pair_id}__{YYYYMMDD}__{direction}	Unique divergence event.
model_version	semantic/project version	xgb_reversion_v1.2.0.
prompt_version	agent prompt schema version	break_agent_v1.1.
controller_version	decision-policy version	controller_v1.0.

B.1 Timestamp conventions

- All ingestion and document timestamps are stored in UTC with timezone information.
- Market trading_date is stored as the US exchange session date.
- SEC acceptance timestamps are converted from SEC source values into UTC but the raw source value is also preserved.
- event_timestamp is an explicit timestamp, not just a date, so evidence cutoffs are unambiguous.
- Every timestamp field name states its meaning: retrieved_at, published_at, accepted_at, generated_at, or event_timestamp. Generic date/time names are avoided.

B.2 Final data contract

The data layer is complete when any displayed model prediction or agent conclusion can be traced backwards from event_id -> model/agent output -> feature/evidence row -> curated source row -> immutable raw file -> external source URL and retrieval metadata, while preserving the event-time information boundary.